Experiment 8: Student Grading Evaluation MapReduce Program (Windows)

Aim

To implement a Java MapReduce program running on a Windows platform to parse student

examination marks and assign corresponding academic letter grades. Procedure
1. Set up your Java and Hadoop environmental path variables within the Windows Command Prompt

to establish clean compiler linking references. 2. Structure your raw input data source values inside a plain text log file following a comma- separated format matching StudentName,Marks. 3. Design the Mapper class logic in Java to split input lines into individual tokens, matching the
student's name as the text key and their marks score as the text value. 4. Design the Reducer class logic to process the collected datasets and run conditional branching
loops to evaluate alpha letter grades based on the numerical scores. 5. Create the Driver configuration routine to bind the custom Mapper and Reducer workflows, define
input and output configurations, and register data format configurations. 6. Compile the Java source files through your Windows command line using a semicolon-delimited (;)
classpath string pointing to your local Hadoop deployment library archives. 7. Compress the compiled byte outputs into a runnable.jar file and trigger execution across your

active storage blocks using the hadoop jar runtime engine wrapper. Main Program (StudentGrader.java)

java

import java.io.IOException;import org.apache.hadoop.conf.Configuration;import org.apache.hadoop.fs.Path;import org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Job;import org.apache.hadoop.mapreduce.Mapper;import org.apache.hadoop.mapreduce.Reducer;import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class StudentGrader {

// Mapper Class
public static class GradeMapper extends Mapper<Object, Text, Text, Text> {
public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
String line = value.toString();
// Split line by comma token delimiter
String[] tokens = line.split(",");

if (tokens.length == 2) {
String studentName = tokens[0].trim();
String marks = tokens[1].trim();
context.write(new Text(studentName), new Text(marks));
}
}
}

// Reducer Class
public static class GradeReducer extends Reducer<Text, Text, Text, Text> {
public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
for (Text val: values) {
try {
int score = Integer.parseInt(val.toString());
String grade;

// Branching conditional evaluation criteria matrix
if (score >= 90) {
grade = "A";
} else if (score >= 80) {
grade = "B";
} else if (score >= 70) {
grade = "C";
} else if (score >= 50) {
grade = "D";
} else {
grade = "F";
}

context.write(key, new Text("Score: " + score + " -> Grade: " + grade));
} catch (NumberFormatException e) {

context.write(key, new Text("Invalid marks format: " + val.toString()));
}
}
}
}

// Driver Entry Point Configuration
public static void main(String[] args) throws Exception {
if (args.length!= 2) {
System.err.println("Usage: StudentGrader <input path> <output path>"); System.exit(-1);
}
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "student grader evaluation"); job.setJarByClass(StudentGrader.class);

job.setMapperClass(GradeMapper.class); job.setReducerClass(GradeReducer.class);

job.setOutputKeyClass(Text.class); job.setOutputValueClass(Text.class);

FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true)? 0: 1);
}
}

Use code with caution. Windows Execution Commands

cmd

:: 1. Compile the source logic targeting native Windows dependencies path mappings

javac -classpath
"%HADOOP_HOME%\share\hadoop\common*;%HADOOP_HOME%\share\hadoop\common\lib*;%HADOOP_H OME%\share\hadoop\mapreduce*" -d wc_classes StudentGrader.java

:: 2. Package output binaries cleanly to a runnable JAR archive

jar -cvf studentgrader.jar -C wc_classes/.

:: 3. Generate mock records via echoing to standard Windows file tree path locations

(echo Alice,92 & echo Bob,65 & echo Charlie,84 & echo David,45) > C:\Users\Public\student_marks.txt

:: 4. Push local source dataset entries straight to core HDFS input channels

hdfs dfs -mkdir -p /grades_input

hdfs dfs -put C:\Users\Public\student_marks.txt /grades_input/

:: 5. Execute processing workload application across runtime layout

hadoop jar studentgrader.jar StudentGrader /grades_input/student_marks.txt /grades_output

:: 6. Display generated grading distribution output logs

hdfs dfs -cat /grades_output/part-r-00000

Use code with caution. Execution & Output

text

C:\hadoop\sbin>hadoop jar studentgrader.jar StudentGrader /grades_input/student_marks.txt /grades_output

26/06/03 14:10:45 INFO mapreduce.Job: Job job_17174000000_0003 completed successfully

C:\hadoop\sbin>hdfs dfs -cat /grades_output/part-r-00000

Alice Score: 92 -> Grade: A

Bob Score: 65 -> Grade: C

Charlie Score: 84 -> Grade: B

David Score: 45 -> Grade: F